

Стандартни типове данни в C#. Масиви в C#

Стандартни типове данни в C#

Типовете данни представляват множества от стойности с еднакви характеристики, като размер в паметта и стойност по подразбиране. В C# те се делят основно на стойностни (value types) и референтни (reference types).

1. Целочислени типове

Те отразяват целите числа и биват със знак и без знак.

- **sbyte**: 8-битов със знак; диапазон [-128...127].
- **byte**: 8-битов без знак; диапазон [0...255].
- **short**: 16-битов със знак; диапазон [-32 768...32 767].
- **ushort**: 16-битов без знак; диапазон [0...65 535].
- **int**: 32-битов със знак; най-често използваният тип за цели числа; диапазон [-2 147 483 648...2 147 483 647].
- **uint**: 32-битов без знак; диапазон [0...4 294 967 295].
- **long**: 64-битов със знак; диапазон [-9 223 372 036 854 775 808...9 223 372 036 854 775 807].
- **ulong**: 64-битов без знак; диапазон [0...18 446 744 073 709 551 615].

2. Реални типове (с плаваща и десетична запетая)

- **float**: 32-битов тип с единична точност (до 7 знака); диапазон $\pm 1.5 \times 10^{-45}$ до $\pm 3.4 \times 10^{38}$.
- **double**: 64-битов тип с двойна точност (15-16 знака); диапазон $\pm 5.0 \times 10^{-324}$ до $\pm 1.7 \times 10^{308}$.
- **decimal**: 128-битов тип с десетична точност (28-29 знака); използва се за финансови пресмятания; диапазон $\pm 1.0 \times 10^{-28}$ до $\pm 7.9 \times 10^{28}$.

3. Други стандартни типове

- **bool**: Булев тип; стойностите са само **true** или **false**.
- **char**: 16-битов символен тип, представящ Unicode символ.
- **string**: Референтен тип, представящ поредица от символи.
- **object**: Базов тип за всички останали типове в .NET.

Операции върху типовете данни

- **Аритметични**: **+**, **-**, *****, **/** (целочислено и реално), **%** (остатък), **++**, **--**.
- **За присвояване**: **=**, **+=**, **-=**, ***=**, **/=**, **%=**.
- **За сравнение**: **==**, **!=**, **>**, **<**, **>=**, **<=**.

- **Логически:** `&&` (И), `||` (ИЛИ), `!` (НЕ), `^` (изключващо ИЛИ).
 - **Побитови:** `&`, `|`, `^`, `~`, `<<`, `>>`.
-

Масиви в C#

Масивът е съвкупност от променливи (елементи) от един и същи тип, разположени в континуална памет и достъпни чрез индекс.

1. Едномерни масиви

Представяват наредена последователност от елементи (вектор).

Синтаксис и инициализация:

```
int[] arr = new int[5]; // Деклариране и заделяне на памет за 5 елемента
int[] numbers = { 1, 2, 3, 4, 5 }; // Непосредствена инициализация
```

2. Многомерни масиви

Най-често използвани са двумерните масиви (матрици), които имат редове и колони.

Синтаксис и инициализация:

```
int[,] matrix = new int[3, 4]; // Матрица с 3 реда и 4 колони
int[,] prefilled = { {1, 2}, {3, 4} }; // Инициализиране на двумерен масив
```

- **Достъп:** Чрез два индекса: `matrix[row, col]`.

3. Назъбени масиви (Jagged arrays)

Това са масиви от масиви, при които всеки ред може да има различна дължина.

Синтаксис и инициализация:

```
int[][] jagged = new int[]; // Масив от 3 масива
jagged = new int[]; // Първият ред е с 5 елемента
jagged = new int[]; // Вторият ред е с 2 елемента
```

- **Памет:** Те представляват съвкупност от референции към отделни обекти-масиви в динамичната памет.

Важно: Индексирането винаги започва от **0**. При опит за достъп извън границите на масива се хвърля изключение `System.IndexOutOfRangeException`. Масивите се съхраняват в динамичната памет (heap), а променливите, които ги достъпват, са референции в стека.

Рекурсивни и итеративни алгоритми. Порядък на операциите

Тема 2 от конспекта обхваща фундаменталните концепции за рекурсия и итерация, като изследва тяхната ефективност и практическо приложение чрез класически примери.

1. Рекурсивни алгоритми (по проф. Голев)

Рекурсията е програмна техника, при която даден метод извиква сам себе си за решаването на определен проблем.

- **Изчисляване на факториел ($n!$):** Проф. Голев дефинира факториел чрез рекурентната формула: $f(n) = n * f(n-1)$ при $f(0) = f(1) = 1$. При всяко рекурсивно извикване се отделя памет в системния стек за локалните променливи. Ако броят на извикванията е твърде голям, това може да доведе до **препълване на стека (stack overflow)**.
- **Числа на Фибоначи:** Дефинират се като $f(n) = f(n-1) + f(n-2)$. Професорът подчертава, че директната рекурсивна реализация е **изключително неефективна**, тъй като едни и същи числа се пресмятат многократно (например $f(2)$ се изчислява много пъти в дървото на извикванията). Броят на стъпките нараства експоненциално с n .

2. Ханойски кули (по Наков)

Задачата за Ханойските кули е класически пример, който се решава най-естествено чрез рекурсия. Тя включва преместването на n диска от един стълб на друг, спазвайки правилото, че по-голям диск не може да бъде поставен върху по-малък.

- **Алгоритъм:** За да се преместят n диска от стълб 1 на стълб 2:
 1. Рекурсивно се преместват горните $n-1$ диска от стълб 1 на помощния стълб 3.
 2. Премества се най-големият (n -ти) диск директно на стълб 2.
 3. Рекурсивно се преместват $n-1$ диска от стълб 3 върху стълб 2.
- **Сложност:** Времето за изпълнение се определя от формулата $T(n) = 2^n - 1$, което представлява **експоненциална сложност**.

3. Сравнение между рекурсия и итерация

Характеристика	Рекурсия	Итерация (Цикли)
Дефиниция	Методът извиква себе си.	Използват се конструкции за повторение (for , while).
Памет	Консумира повече ресурси поради заделяне на стек рамки за всяко извикване.	По-ефективна откъм памет; работи в рамките на една стек рамка.

Четимост	Води до по-елегантни и кратки решения за математически дефинирани проблеми.	Може да бъде по-сложна за дефиниране при специфични алгоритми.
Производителност	Често по-бавна поради излишни пресмятания (ако не се ползва мемоизация).	Обикновено по-бърза и с линейна скорост при прости задачи.

4. Порядък на операциите (Сложност)

Порядъкът на операциите (сложността) определя как нараства броят на стъпките спрямо обема на входните данни (n).

- **Факториел:** И при двете реализации сложността е **линейна $O(n)$** , тъй като се извършват n умножения.
- **Числа на Фибоначи:**
 - Рекурсивно: **Експоненциална сложност $O(2^n)$** или по-точно $O(\phi^n)$, което прави алгоритъма неизползваем за големи n .
 - Итеративно: **Линейна сложност $O(n)$** , изискваща само един цикъл до n .
- **Ханойски кули:** Сложността е неизбежно **експоненциална $O(2^n)$** , тъй като броят на преместванията се удвоява при всеки нов диск.

Заключение: Проф. Голев съветва рекурсията да се избягва при линейни изчислителни процеси (като факториел) и да се предпочита итерацията, освен ако рекурсивното решение не е значително по-ясно и не води до прекомерна консумация на ресурси.

Сортировки: селекция, мехурче и вмъкване

Ето подробно изложение на Тема 3 от конспекта, базирано на учебните материали на проф. Ангел Голев, с допълнителни детайли за сложността от книгата на Наков и другите теоретични източници.

За реализациите на проф. Голев е важно да се отбележи, че той използва помощен метод за размяна на елементи в масив, дефиниран като разширяващ метод (extension method):

```
static class IntArraySwap
{
    static public void SwapElements(this int[] array, int indexOne, int indexTwo)
    {
        int tmp = array[indexOne];
        array[indexOne] = array[indexTwo];
        array[indexTwo] = tmp;
    }
}
```

1. Сортировка чрез пряка селекция (Selection Sort)

Идея на алгоритъма: На всяка стъпка се намира минималният елемент в оставащата несортирана част от масива и се разменя с елемента на текущата първа позиция в тази част. На стъпка i намираме минималния елемент от позиция $i-1$ до $n-1$ и го поставяме на позиция $i-1$. Процесът се повтаря $n-1$ пъти.

С# код от помагалото на проф. Голев:

```
static void SelectionSort(int[] arr)
{
    int i, j, pos;
    int n = arr.Length;
    int min_el;
    for (i = 0; i < n - 1; i++)
    {
        min_el = arr[i];
        pos = i;
        for (j = i + 1; j < n; j++)
        {
            if (arr[j] < min_el)
            {
                min_el = arr[j];
                pos = j;
            }
        }
        arr.SwapElements(i, pos);
    }
}
```

Оценка на порядъка на операциите:

- **Най-лош случай:** $O(n^2)$.
- **Среден случай:** $O(n^2)$.
- **Най-добър случай:** $O(n^2)$. При селекцията винаги се извършват точно $n(n-1)/2$ сравнения, независимо дали масивът е сортиран или не.

2. Метод на мехурчето (Bubble Sort)

Идея на алгоритъма: Последователно се сравняват всеки два съседни елемента. Ако първият е по-голям от втория, те си разменят местата. След всяко пълно преминаване през масива най-големият елемент "изплува" най-вдясно. Сортировката приключва, когато при поредното преминаване не е извършена нито една размяна.

С# код от помагалото на проф. Голев:

```
static void BubbleSort(int[] arr)
{
    bool exchange;
    int i, k;
    int n = arr.Length;
    for (k = n - 1; k > 0; k--)
    {
        exchange = false;
        for (i = 0; i < k; i++)
            if (arr[i] > arr[i + 1])
            {
                arr.SwapElements(i, i + 1);
                exchange = true;
            }
        if (!exchange) break;
    }
}
```

Оценка на порядъка на операциите:

- **Най-лош случай:** $O(n^2)$.
 - **Среден случай:** $O(n^2)$.
 - **Най-добър случай:** $O(n)$. Благодарение на проверката **exchange**, ако масивът е вече сортиран, алгоритъмът ще спре след първото обхождане.
-

3. Сортировка чрез вмъкване (Insertion Sort)

Идея на алгоритъма: Приема се, че елементите до индекс k са вече сортирани. Взима се елементът на позиция k и се сравнява с тези вляво от него. Докато текущият елемент е по-малък от съседите си вляво, те се изместват с една позиция надясно, за да се освободи място, където новият елемент да бъде "вмъкнат".

С# код от помагалото на проф. Голев:

```
static void InsertionSort(int[] arr)
```

```

{
    int i, k;
    int x;
    int n = arr.Length;
    for (k = 1; k < n; k++)
    {
        x = arr[k];
        i = k - 1;
        while (i >= 0 && x < arr[i])
            arr[i + 1] = arr[i--];
        arr[i + 1] = x;
    }
}

```

Оценка на порядъка на операциите:

- **Най-лош случай:** $O(n^2)$.
- **Среден случай:** $O(n^2)$.
- **Най-добър случай:** $O(n)$. Когато масивът е вече сортиран, вътрешният цикъл `while` не се изпълнява нито веднъж и сложността е линейна.

Обобщение за сложността (по Наков и Wikipedia):

Алгоритъм	Най-добър	Среден	Най-лош
	p		
Selection Sort	$O(n^2)$	$O(n^2)$	$O(n^2)$
Bubble Sort	$O(n)$	$O(n^2)$	$O(n^2)$
Insertion Sort	$O(n)$	$O(n^2)$	$O(n^2)$

Професор Голев подчертава, че и трите метода са с квадратичен порядък $O(n^2)$ и са подходящи само за масиви с малък брой елементи. Методът на вмъкването и методът на мехурчето са **устойчиви** (stable) сортировки, докато пряката селекция **не е устойчива**.

Бърза сортировка (порядък на операциите)

Ето подробно изложение на **Тема 4 „Бърза сортировка (порядък на операциите)“**, базирано изцяло на учебните материали на **проф. Ангел Голев**.

1. Идея на алгоритъма Quick Sort

Бързата сортировка е един от най-често използваните методи в практиката, тъй като е лесен за реализация и не изисква сложни операции. Алгоритъмът следва принципа „**разделяй и владей**“ и се състои от следните стъпки:

- **Избор на стопиращ елемент (pivot):** Избира се елемент x от подмасива (обикновено средният, но може да бъде и първият, последният или случаен).
- **Разделяне (Partitioning):** Масивът се разделя на две части спрямо стойността x . В лявата част се преместват всички елементи, които са по-малки или равни на x , а в дясната – тези, които са по-големи или равни на x . Това се постига с два индекса, започващи от двата края, които се движат един към друг и разменят елементи, които са в грешната половина.
- **Рекурсия:** Алгоритъмът се прилага рекурсивно за лявата и за дясната част на масива, докато се достигнат подмасиви с един елемент, които вече са сортирани.

2. Реализация на C# (код от проф. Голев)

Професорът използва разширяващ метод за размяна на елементи (**SwapElements**), дефиниран в неговото помагало. Самият метод за сортиране приема масива и неговите лява (**lb**) и дясна (**rb**) граница:

```
static void QuickSort(int[] arr, int lb, int rb)
{
    int i = lb, j = rb;
    int x;

    // Избор на средния елемент за pivot
    x = arr[(lb + rb) / 2];

    while (i <= j) // Разделяне на масива на две части
    {
        while (i <= rb && arr[i] < x) i++;
        while (lb <= j && arr[j] > x) j--;

        if (i <= j)
        {
            // Размяна на елементите и преместване на индексите
            arr.SwapElements(i, j);
        }
    }
}
```



```

        i++;
        j--;
    }
}

// Рекурсивни извиквания за двете части
if (i < rb) arr = QuickSort(arr, i, rb);
if (lb < j) arr = QuickSort(arr, lb, j);

return arr;
}

**

```

3. Оценка на порядъка на операциите (Сложност)

Времовата сложност на Quick Sort зависи от това колко балансирано се разделя масивът на всяка стъпка:

- **Най-добър случай: $O(n * \log n)$** Това се постига, когато на всеки ход масивът се разделя точно на две равни части. В този случай дълбочината на рекурсията е $\log n$, а на всяко ниво се извършват общо n операции по сравнение и размяна.
- **Среден случай: $O(n * \log n)$** На практика Quick Sort е изключително бърз и в средния случай работи със същия логаритмичен порядък, като често изпреварва други алгоритми поради ниските си константи.
- **Най-лош случай: $O(n^2)$** Случва се, когато разделянето е максимално небалансирано (например едната част съдържа само един елемент, а другата – всички останали). Това обикновено става при вече сортиран масив и лош избор на *pivot* (например първия или последния елемент). В този случай броят на стъпките е пропорционален на квадрата на броя елементи.

Оптимизация: Проф. Голев отбелязва, че скоростта може да се подобри, като при малък брой елементи в подмасива рекурсията се прекратява и се прилага по-прост метод като мехурче или вмъкване.

Двоично търсене в сортиран масив и сортировка чрез сливане

Ето подробно изложение на **Тема 5 „Двоично търсене в сортиран масив и сортировка чрез сливане“**, базирано на материалите от проф. Ангел Голев и книгата на Светлин Наков.

1. Двоично търсене в сортиран масив (Binary Search)

Идея на алгоритъма: Двоичното търсене е високоефективен метод за намиране на елемент в **предварително сортиран** масив. Алгоритъмът използва принципа „разделяй и владей“:

1. Определят се лява (l) и дясна (r) граница на търсения интервал.
2. Намира се средният елемент m .
3. Сравнява се търсената стойност със средния елемент:
 - Ако са равни, елементът е намерен.
 - Ако търсената стойност е по-малка, търсенето продължава в лявата половина ($r = m - 1$).
 - Ако е по-голяма, търсенето продължава в дясната половина ($l = m + 1$).
4. Процесът се повтаря, докато елементът бъде намерен или интервалът се изчерпи ($l > r$).

С# код от помагалото на проф. Голев: Професорът представя реализация на двоично търсене в масив от низове (речник):

```
int FindWord(string str)
{
    int l, r, m, cmp;
    l = 0; r = dict.Length - 1;

    while (l <= r)
    {
        m = (l + r) / 2;
        cmp = str.CompareTo(dict[m]);
        if (cmp == 0)
            return m; // Намерен
        else if (cmp < 0)
            r = m - 1; // Търсене вляво
        else
            l = m + 1; // Търсене вдясно
    }
    return -1; // Не е намерен
}
```

2. Сортировка чрез сливане (Merge Sort)

Идея на алгоритъма: Това е алгоритъм със сложност $O(n * \log n)$, който разделя масива на две равни части, сортира ги рекурсивно и накрая ги слива в един общ сортиран масив.

Поради разделянето винаги на две, дълбочината на рекурсията е $\log n$, а операцията по сливане на всяко ниво е пропорционална на n .

С# код от помагалото на проф. Голев: Реализацията изисква помощен масив `tmp_arr` за временно съхранение на данните при сливането.

```
static int[] tmp_arr; // Помощен масив

static void MergeArr(int[] arr, int left_i, int middle_i, int right_i)
{
    int i, j, k;
    // Преместване на първата част във временен масив
    i = 0; j = left_i;
    while (j <= middle_i) tmp_arr[i++] = arr[j++];

    // Сливане на масивите
    i = 0; j = middle_i + 1; k = left_i;
    while (k < j && j <= right_i)
        if (tmp_arr[i] <= arr[j])
            arr[k++] = tmp_arr[i++];
        else
            arr[k++] = arr[j++];

    // Добавяне на останалите елементи
    while (k < j) arr[k++] = tmp_arr[i++];
}

static void MergeSort(int[] arr, int left_i, int right_i)
{
    int middle_i;
    if (left_i < right_i)
    {
        middle_i = (left_i + right_i) / 2;
        MergeSort(arr, left_i, middle_i); // Сортиране на лява част
        MergeSort(arr, middle_i + 1, right_i); // Сортиране на дясна част
        MergeArr(arr, left_i, middle_i, right_i); // Сливане
    }
}
```

3. Устойчивост на сортировките (Stability)

Концепция: Една сортировка се нарича **устойчива (stable)**, ако запазва относителния ред на елементите с еднакви ключове. Ако в оригиналния списък запис R се появява преди запис S и двата имат еднакви ключове, при устойчива сортировка R винаги ще бъде преди S и в изходния списък. Това е критично при сортиране по няколко критерия последователно (например първо по име, после по отдел).

Класификация на разгледаните сортировки:

Алгоритъм	Устойчивост	Обяснение
Selection Sort (Селекция)	Неустойчива	При размяната на минималния елемент с текущия може да се прескочат и разместят равни елементи.
Bubble Sort (Мехурче)	Устойчива	Разменя елементи само ако единият е строго по-голям, запазвайки реда на равните.
Insertion Sort (Вмъкване)	Устойчива	Елементите се вмъкват след равните на тях, запазвайки реда.
Quick Sort (Бърза)	Неустойчива	Процесът на разделяне (partitioning) около пивота обикновено разменя отдалечени елементи и нарушава стабилността.
Merge Sort (Сливане)	Устойчива	При сливането, ако два елемента са равни, се взима първо този от лявата (оригинално по-предната) част.

Извод: Проф. Голев отбелязва, че устойчивостта е важна, когато се налагат последователни сортировки по различни признаци. За обща употреба **Merge Sort** е предпочитан, когато се изисква стабилност и висока скорост, докато **Quick Sort** е по-бърз в средния случай, но е нестабилен.

Обхождане с връщане назад

Въз основа на предоставените източници, ето подробно изложение на **Тема 6 „Обхождане с връщане назад“**, като е използвана теоретичната част от книгата на проф. Ангел Голев и реализацията на задачите от неговото помагало и книгата на Наков (която допълва липсващите в откъсите на проф. Голев реализации за лабиринт).

1. Теоретична същност на метода Backtracking (по проф. Голев)

Проф. Голев дефинира **обхождането (търсене) с връщане назад (Backtracking)** като метод, при който решението на дадена задача се конструира последователно.

Основни принципи:

- **Последователно изграждане:** На всяка стъпка се прави опит да се разшири текущото частично решение така, че то да отговаря на условията на задачата.
 - **Продължаване при успех:** Ако разширяването е успешно, алгоритъмът преминава към следващата стъпка.
 - **Връщане назад (Backtrack):** Когато се изчерпят всички възможности за разширяване на текущата стъпка и нито една не води до решение, алгоритъмът се връща на предходната стъпка, за да пробва друга алтернатива.
 - **Ефективност:** Професорът отбелязва, че този метод е незаменим за задачи, при които генерирането на всички възможни варианти (пълно изчерпване) е невъзможно в реално време (например задачата за дамите, където вариантите са 8^8).
-

2. Път в лабиринт (по материалите в източниците)

Алгоритъмът за намиране на път в лабиринт е класическо приложение на метода. Тъй като в откъсите от помагалото на проф. Голев тази задача е спомената като пример, но кодът е наличен в книгата на Наков (също част от източниците), представям реализацията от там, която следва същите принципи на Backtracking.

Идея на алгоритъма:

1. Започва се от стартова позиция (0,0).
2. Ако текущата клетка е изходът ('e'), пътът е намерен.
3. Ако клетката е непроходима ('*') или вече посетена ('s'), се връщаме назад.
4. Маркираме текущата клетка като посетена.
5. Рекурсивно опитваме четирите посоки (наляво, нагоре, надясно, надолу).
6. **Важно за Backtracking:** След връщане от рекурсията, маркираме клетката обратно като свободна, за да може тя да участва в други възможни пътища.

С# код на метода:

```
static void FindPath(int row, int col, char direction)
{
    if ((col < 0) || (row < 0) || (col >= lab.GetLength(1)) || (row >= lab.GetLength(0)))
    {
        return; // Извън границите
    }

    path[position] = direction;
    position++;

    if (lab[row, col] == 'e') // Проверка за изход
    {
```

```

    PrintPath(path, 1, position - 1);
}

if (lab[row, col] == ' ')
{
    lab[row, col] = 's'; // Маркиране като посетена

    FindPath(row, col - 1, 'L'); // ляво
    FindPath(row - 1, col, 'U'); // нагоре
    FindPath(row, col + 1, 'R'); // надясно
    FindPath(row + 1, col, 'D'); // надолу

    lab[row, col] = ' '; // Връщане назад (Backtrack)
}
position--;
}

```

3. Задачата за дамите

Проф. Голев споменава задачата за дамите като типичен пример, при който Backtracking е единственото решение, но в предоставените откъси от Глава 4 той подробно разписва **Задачата за разходката на коня**, която е идентична като логика на обхождане.

Алгоритъмът (по аналогия с реализираната от професора „Разходка на коня“):

- Маркираме текущата позиция като заета.
- Ако всички фигури са поставени (всички позиции посетени) -> решение.
- Опитваме последователно всички допустими движения.
- Ако обхождането на следваща позиция не доведе до решение, **маркираме текущата позиция като свободна (visited = 0)** и се връщаме назад.

C# код на логиката за връщане назад (от решението на професора):

```

public bool Solve(int x, int y, int current)
{
    visited[x, y] = current; // Маркираме позицията

    if (AllNodesVisited()) return true; // Успех

    for (int i = 0; i < offsetX.Length; i++)
    {
        int newX = x + offsetX[i];
        int newY = y + offsetY[i];
    }
}

```

```

        if (IsSafe(newX, newY)) // Проверка на условията
        {
            if (Solve(newX, newY, current + 1))
                return true;
        }
    }

    visited[x, y] = 0; // СЪПКА НАЗАД (Backtracking)
    return false;
}

```

Заклучение: Методът разчита на системния стек (чрез рекурсия), за да пази състоянието на частичното решение и да позволява отказ от „грешни“ пътеки по време на търсенето.

Линеен списък. Реализация и операции. Реализация в .NET

Ето подробно изложение на **Тема 7 „Линеен списък. Реализация и операции. Реализация в .NET“**, изготвено въз основа на Глава 16 от книгата на Светлин Наков и предоставените материали от Microsoft.

1. Теоретична същност

Линейният списък е абстрактна структура от данни (АСД), която представлява **наредена последователност от елементи**. Основните му характеристики са:

- **Наредба:** Между елементите съществува линейна зависимост – всеки елемент (освен първия и последния) има точно един предходник и един наследник.
- **Дължина:** Броят на елементите в списъка може да се променя динамично по време на изпълнение на програмата.
- **Гъвкавост:** Позволява вмъкване и изтриване на елементи на всяка една позиция, както и последователното им обхождане.

В C# абстракцията на списъка се дефинира чрез интерфейса **`ICollection<T>`**, който задава стандартните операции, без да се интересува от конкретната им реализация.

2. Реализации на линеен списък

В практиката съществуват два основни начина за реализация на списъци:

A. Статичен списък (чрез масив)

Реализира се чрез **вътрешен масив**, който автоматично се преоразмерява (обикновено удвоява капацитета си), когато се напълни.

- **Предимство:** Осигурява светкавичен достъп до елементите по индекс.
- **Недостатък:** Вмъкването и изтриването във вътрешността на списъка са бавни, тъй като изискват изместване на всички следващи елементи.

Б. Свързан списък (динамична реализация)

Състои се от отделни елементи (**възли**), където всеки възел пази своята стойност и **референция (указател)** към следващия (и предходния при двойно свързаните списъци) елемент.

- **Предимство:** Много бързо добавяне и изтриване в двата края на списъка, тъй като се променят само указатели.
- **Недостатък:** Бавен достъп по индекс, защото изисква последователно обхождане от началото на списъка.

3. Операции и тяхната сложност

Операция	List<T> (Масив)	LinkedList<T> (Свързан)	Описание
Достъп по индекс	O(1)	O(N)	Директен при масива; изисква обхождане при свързания списък.
Добавяне (в края)	Амортизирано O(1)	O(1)	При List<T> е бързо, освен при преоразмеряване.
Вмъкване (на позиция)	O(N)	O(1)	При LinkedList е константно, ако вече имаме референция към мястото.
Изтриване (по индекс)	O(N)	O(N)	Изисква изместване при масива или намиране на елемента при свързания списък.
Търсене (по стойност)	O(N)	O(N)	Изисква последователно обхождане и сравняване на всеки елемент.

4. Реализация в .NET (System.Collections.Generic)

.NET платформата предоставя две основни имплементации в пространството от имена **System.Collections.Generic**:

- **List<T>**: Най-често използваната структура. Представява динамично-разширяем масив. Подходящ е, когато се нуждаем от **бърз достъп по индекс** и добавяне предимно в края на списъка.
- **LinkedList<T>**: Реализация на **двусвързан списък**. Всеки елемент е обект от тип **LinkedListNode<T>**, съдържащ свойства **Value**, **Next** и **Previous**. Подходящ е при чести операции по **вмъкване и изтриване на произволни места**.

Важно за търсенето: И двата класа поддържат линейно търсене чрез метода **Contains()** или **IndexOf()**. За **List<T>** е наличен и методът **BinarySearch()**, който работи със сложност **O(log N)**, но само ако списъкът е предварително сортиран.

Стек и опашка. Реализация и операции. Реализация в .NET

Тема 8 от конспекта разглежда две от най-фундаменталните линейни структури от данни – стек и опашка, техните реализации и практическото им приложение.

1. Стек (Stack)

Теоретична същност: Стекът е структура от данни с поведение **LIFO (Last-In-First-Out)** – „последен влязъл, пръв излязъл“. Може да се визуализира като купчина чинии, при която добавянето и премахването стават само от най-горната страна.

Основни операции:

- **Push(T)**: Добавя нов елемент на върха на стека.
- **Pop()**: Връща и премахва елемента от върха на стека.
- **Peek()**: Връща най-горния елемент без да го премахва.
- **Count**: Връща текущия брой на елементите в стека.

2. Опашка (Queue)

Теоретична същност: Опашката е структура с поведение **FIFO (First-In-First-Out)** – „пръв влязъл, пръв излязъл“. Тя моделира реална опашка от хора, където елементите се добавят в края и се извличат от началото.

Основни операции:

- **Enqueue(T)**: Добавя елемент в края на опашката.
 - **Dequeue()**: Връща и премахва елемента от началото на опашката.
 - **Peek()**: Връща елемента от началото без да го премахва.
 - **Count**: Връща броя на елементите в опашката.
-

3. Сравнение на реализациите

Характеристика	Реализация с масив (статична)	Реализация със свързан списък (динамична)
Памет	Изисква предварително заделяне на капацитет. Когато се напълни, масивът се преоразмерява (обикновено се удвоява), което е бавна операция.	Заделя памет динамично за всеки нов елемент. Изисква допълнително място за указатели (пойнтъри) към съседните елементи.
Ефективност	Изключително бърз достъп до елементите и по-добра кеш-ефективност.	По-гъвкава при неизвестен брой елементи, избягва тежките преоразмерявания на масива.
Специфика	При опашките се използва зациклен масив (circular array) и модулно индексирание, за да се избегне постоянното изместване на елементи при изтриване.	Опашката поддържа два указателя – към началото (head) и към края (tail) за бързо добавяне и извличане.

4. Реализация в .NET (**System.Collections.Generic**)

В .NET Framework тези структури са вградени като стандартни шаблонни (Generic) класове:

- **Stack<T>**: Реализиран е **статично чрез масив**, който се преоразмерява автоматично при нужда.
 - **Queue<T>**: Също използва **вътрешен зациклен масив**, като позволява задаване на начален капацитет.
 - **LinkedList<T>**: Може да се използва за ръчна имплементация на опашка или стек, тъй като позволява бързо добавяне и изтриване в двата си края.
-

5. Постфиксен запис и оценка със стек

Постфиксният запис (известен като **Обратен полски запис**) е начин за представяне на математически изрази, при който операторът стои **след** своите операнди (например $2\ 3\ +$ вместо $2 + 3$).

Предимства:

- Не се налага използването на **скоби** за указване на приоритет.
- Изразът се обработва еднопосочно (отляво надясно).

Алгоритъм за оценка със стек:

1. Обхождаме елементите на израза един по един.
2. Ако елементът е **число (операнд)**, го добавяме (Push) в стека.
3. Ако елементът е **операция** (напр. $+$, $*$):
 - Изваждаме (Pop) последните две стойности от стека.
 - Изпълняваме операцията върху тях (внимава се за реда – първият изваден е втори операнд).
 - Добавяме (Push) резултата обратно в стека.
4. След края на израза, единственият останал елемент в стека е неговата **крайна стойност**.

Хеш таблици. Реализация и операции. Реализация в .NET

Ето подробно изложение на **Тема 9 „Хеш таблици. Реализация и операции**.

Реализация в .NET“, подготвено въз основа на Глава 18 от книгата на Светлин Наков и материалите от Microsoft Learn.

1. Същност на хеширането

Хеширането е процес на преобразуване на ключ (обект от произволен тип) в числова стойност (индекс), която се използва за съхранение и търсене на данни в масив.

- **Хеш-таблица:** Това е структура от данни, реализирана чрез масив, в която елементите (наредени двойки ключ-стойност) са разположени на позиции, определени от техните ключове.
- **Хеш-код и Хеш-функция:** Хеш-функцията приема ключ и връща **хеш-код**. Индексът в масива се изчислява чрез формулата: $index = hash(key) \% capacity$, където **capacity** е размерът на масива.
- **Load Factor (Степен на запълненост):** Това е съотношението между броя на заетите елементи и капацитета на таблицата. Обикновено, когато запълването надвиши определен праг (напр. 0.72 или 0.75), таблицата се преоразмерява (разширява) и всички елементи се преподреждат (rehash).

2. Основни операции

Всички основни операции в хеш-таблицата разчитат на бързото изчисляване на индекса по ключ.

- **Включване (Add/Set)**: Изчислява се хеш-кодът на ключа, намира се индексът и стойността се записва на тази позиция. Ако ключът вече съществува, старата стойност се подменя.
- **Търсене (Get/ContainsKey)**: Прилага се хеш-функцията върху ключа, за да се намери неговата позиция в масива, и се извлича стойността. Тази операция е изключително бърза, тъй като не изисква обхождане на целия масив.
- **Изключване (Remove)**: Намира се позицията на ключа и съответната двойка се премахва от таблицата.

3. Реализация в .NET

В .NET платформата има две основни имплементации на хеш-таблицы:

- **Dictionary<TKey, TValue>**: Модерната, шаблонна (generic) имплементация. Тя е силно типизирана и осигурява най-висока производителност.
- **Hashtable**: По-стара имплементация, която работи с обекти (**object**) и изисква постоянно кастване (boxing/unboxing).

4. Колизии и справяне с тях

Колизия възниква, когато два различни ключа произвеждат един и същ хеш-код или се картографират към един и същ индекс в масива. Според източниците има два основни метода за справяне с тях:

- **Нареждане в списък (Chaining)**: Елементите с еднакви хеш-кодове се съхраняват в **свързан списък** на съответната позиция в масива. Тази стратегия се използва от класа **Dictionary<TKey, TValue>**.
- **Отворена адресация / Rehashing**: При колизия се търси друга свободна позиция в самия масив чрез определен алгоритъм (напр. линейно пробване или двойно хеширане). Класът **Hashtable** в .NET използва **double hashing (rehashing)**.

5. Big-O сложност на операциите

Хеш-таблиците са проектирани за максимална бързина при работа с ключове.

- **Среден случай**: Търсенето, добавянето и изтриването имат константна сложност **O(1)**. Това означава, че времето за достъп не зависи от броя на елементите в речника.

- **Най-лош случай:** При лоша хеш-функция, която води до много колизии, сложността може да достигне до линейна $O(n)$. В такъв случай хеш-таблицата се изразжда в линеен списък.
- **Разширяване:** Операцията по разширяване на капацитета е бавна ($O(n)$), но се извършва рядко, поради което амортизираната сложност на добавянето остава $O(1)$.

Дървета, двоични дървета. Реализация. Обхождане

Тема 10 обхваща йерархичните структури от данни, като се фокусира върху тяхната дефиниция, програмна реализация и методите за посещение на всеки техен елемент.

1. Реализация на дърво в C#

Дървото е рекурсивна структура от данни, състояща се от върхове, свързани с ребра. Според материалите на Наков и проф. Голев, най-естественият начин за реализация е чрез създаване на клас за отделния възел (връх) и клас за самото дърво.

А. Общо дърво (с произволен брой наследници)

Всеки връх съдържа своята стойност и списък от референции към своите наследници.

C# реализация (по Наков):

```
public class TreeNode<T> {
    public T Value { get; set; }
    private List<TreeNode<T>> children; // Списък от наследници

    public TreeNode(T value) {
        this.Value = value;
        this.children = new List<TreeNode<T>>();
    }

    public void AddChild(TreeNode<T> child) {
        this.children.Add(child); // Добавяне на дете
    }
}

public class Tree<T> {
    public TreeNode<T> Root { get; private set; } // Корен на дървото
    public Tree(T value) {
        this.Root = new TreeNode<T>(value);
    }
}
```

```
}
```

2. Реализация на двоично дърво в C#

Двоичното дърво е дърво, в което всеки връх има най-много два наследника – **ляв** и **десен**. За разлика от общото дърво, тук наредбата на наследниците е от съществено значение.

C# реализация (по Наков и проф. Голев): Проф. Голев подчертава, че елементите на двоичните дървета обикновено са инстанции от клас, съдържащ ляв и десен наследник.

```
public class BinaryTree<T> {  
    public T Value { get; set; }  
    public BinaryTree<T> LeftChild { get; set; } // Ляв наследник  
    public BinaryTree<T> RightChild { get; set; } // Десен наследник  
  
    public BinaryTree(T value, BinaryTree<T> left = null, BinaryTree<T> right = null) {  
        this.Value = value;  
        this.LeftChild = left;  
        this.RightChild = right;  
    }  
}
```

3. Обхождане на двоично дърво

Обхождането е процес на посещаване на всеки връх точно веднъж. Трите основни вида обхождане в дълбочина (DFS) се различават по момента, в който се обработва коренът спрямо неговите поддървета.

1. Preorder (Корен-Ляво-Дясно / КЛД)

- **Дефиниция:** Първо се посещава коренът, след това рекурсивно се обхожда лявото поддърво и накрая дясното.
- **Алгоритъм:**
 1. Посети текущия възел.
 2. Посети левия наследник.
 3. Посети десния наследник.

Код:

```
public void Preorder(BinaryTree<T> node) {  
    if (node != null) {
```

```

        Console.Write(node.Value + " "); // 1. Корен
        Preorder(node.LeftChild);      // 2. Ляво
        Preorder(node.RightChild);     // 3. Дясно
    }
}

```

•

2. Inorder (Ляво-Корен-Дясно / ЛКД)

- **Дефиниция:** Първо се обхожда лявото поддърво, след това се посещава коренът и накрая дясното поддърво. При двоичните дървета за търсене това обхождане извежда елементите в нарастващ ред.
- **Алгоритъм:**
 1. Посети левия наследник.
 2. Посети текущия възел.
 3. Посети десния наследник.

Код:

```

public void Inorder(BinaryTree<T> node) {
    if (node != null) {
        Inorder(node.LeftChild);      // 1. Ляво
        Console.Write(node.Value + " "); // 2. Корен
        Inorder(node.RightChild);     // 3. Дясно
    }
}

```

•

3. Postorder (Ляво-Дясно-Корен / ЛДК)

- **Дефиниция:** Първо се обхожда лявото поддърво, след това дясното и накрая се посещава коренът.
- **Алгоритъм:**
 1. Посети левия наследник.
 2. Посети десния наследник.
 3. Посети текущия възел.

Код:

```

public void Postorder(BinaryTree<T> node) {
    if (node != null) {
        Postorder(node.LeftChild);    // 1. Ляво
        Postorder(node.RightChild);   // 2. Дясно
        Console.Write(node.Value + " "); // 3. Корен
    }
}

```

-

Заклучение: Проф. Голев използва подобна рекурсивна логика за изчисляване на аритметични изрази, записани в дърво, където операцията (коренът) се изпълнява върху резултатите от поддърветата. И трите метода на обхождане имат **линейна сложност $O(n)$** , тъй като всеки връх се посещава точно веднъж.

Двоични дървета за търсене

Тема 11 от конспекта разглежда **наредените двоични дървета за търсене (Binary Search Trees - BST)**, които са специализиран вид двоични дървета, проектирани за ефективно съхранение и бързо намиране на данни.

1. Основно свойство на двоичното дърво за търсене

Основното свойство на BST е, че за всеки връх **n** е изпълнено следното:

- Всички ключове в **лявото поддърво** на върха са **по-малки** от неговия ключ.
- Всички ключове в **дясното поддърво** на върха са **по-големи** от неговия ключ.
Това свойство гарантира, че елементите в дървото са индиректно сортирани, което позволява изключително бързо търсене.

2. Операция „Търсене“ (Search)

Алгоритъм: Търсенето започва от корена. Ако търсената стойност е равна на текущия ключ, елементът е намерен. Ако е по-малка, търсенето продължава в лявото поддърво; ако е по-голяма – в дясното. Процесът се повтаря до намиране на елемента или достигане на празен наследник (NIL).

C# код (итеративна реализация по Наков):

```
private BinaryTreeNode<T> Find(T value) {  
    BinaryTreeNode<T> node = this.root;  
    while (node != null) {  
        int compareTo = value.CompareTo(node.Value);  
        if (compareTo < 0) {  
            node = node.LeftChild; // Продължаваме вляво  
        } else if (compareTo > 0) {  
            node = node.RightChild; // Продължаваме вдясно  
        } else {  
            return node; // Намерен  
        }  
    }  
}
```



```
    return null; // Не съществува
}
```

Времева сложност: В средния случай е $O(\log n)$, а в най-лошия (при изродено дърво във вид на списък) е $O(n)$.

3. Операция „Включване“ (Insertion)

Алгоритъм: Новите елементи винаги се добавят като **листа**. Алгоритъмът следва пътя на търсенето, за да намери подходящата позиция, която поддържа основното свойство на BST.

C# код (рекурсивна реализация по Наков):

```
public void Insert(T value) {
    if (value == null) throw new ArgumentNullException();
    this.root = Insert(value, null, root);
}

private BinaryTreeNode<T> Insert(T value, BinaryTreeNode<T> parentNode,
BinaryTreeNode<T> node) {
    if (node == null) {
        node = new BinaryTreeNode<T>(value);
        node.Parent = parentNode;
    } else {
        int compareTo = value.CompareTo(node.Value);
        if (compareTo < 0) node.LeftChild = Insert(value, node, node.LeftChild);
        else if (compareTo > 0) node.RightChild = Insert(value, node, node.RightChild);
    }
    return node;
}
```

Времева сложност: Пропорционална на височината на дървото – $O(h)$. При балансирано дърво е $O(\log n)$, при небалансирано – $O(n)$.

4. Операция „Изключване“ (Deletion)

Това е най-сложната операция, при която се разглеждат три случая:

1. **Връх без наследници (листо):** Директно се изтрива, като референцията на родителя се насочва към `null`.
2. **Връх с един наследник:** Изтриваният връх се замества от своя наследник (детето му се „изкачва“ на неговото място).
3. **Връх с два наследника:** Намира се неговият **наследник по стойност** (най-малкият елемент в дясното му поддърво), стойностите им се разменят и тогава се изтрива вече преместеният елемент (който е в първи или втори случай).

С# код (фрагмент за два наследника по Наков):

```
if (node.LeftChild != null && node.RightChild != null) {  
    BinaryTreeNode<T> replacement = node.RightChild;  
    while (replacement.LeftChild != null) {  
        replacement = replacement.LeftChild; // Намиране на най-малкия вдясно  
    }  
    node.Value = replacement.Value; // Размяна на стойностите  
    node = replacement; // Подготовка за изтриване на листа  
}
```

Времева сложност: $O(h)$, което е $O(\log n)$ при балансирано дърво.

Важно: Проф. Голев и Наков отбелязват, че ако в BST се добавят елементи в сортиран ред, дървото се изражда в списък и операциите губят своята ефективност ($O(n)$). За решаване на този проблем се използват **балансиран дървета** (като червено-черни или AVL), които гарантират сложност $O(\log n)$.

Графи. Реализация. Матрица на съседства и свойства

В Глава 6 от своето учебно помагало **проф. Ангел Голев** разглежда структурата от данни граф, нейните реализации и математически свойства.

1. Дефиниция за структурата Граф

Според професора, графът представлява **множество от върхове (nodes) и свързващите ги ребра (edges)**. В зависимост от естеството на връзките, графите биват:

- **Неориентиран граф:** Ребрата са ненаредени двойки върхове.
- **Ориентиран граф:** Ребрата са наредени двойки върхове.
- **Претеглен граф:** Към всяко ребро е съпоставено тегло (например дължина или цена).

2. Начини за представяне на граф

Професорът посочва три основни начина за програмна реализация:

1. **Списък от двойки върхове (ребра):** Представя се чрез списък от наредени или ненаредени двойки $\{u, v\}$, които определят наличните връзки.
2. **Списък на наследниците:** Във всеки връх се съдържа списък с указатели или индекси към върховете, към които сочат ребрата, започващи от него.
3. **Матрица на съседство:** Квадратна матрица, която описва връзките между всеки два върха.

3. Матрица на съседство

Матрицата на съседство е квадратна матрица с размер $n \times n$, където n е броят на върховете на графа.

- **Елементът $g[i, j]$:** Представя реброто от връх i до връх j .
- **Стойности:** Ако се интересуваме само от наличието на връзка, стойностите са **0** или **1**. При претеглени графи елементите съдържат **теглото на реброто**.
- **Симетрия:** При неориентиран граф матрицата винаги е **симетрична** спрямо главния диагонал.

4. Свойства на матрицата на съседство

Проф. Голев изтъква следните ключови теоретични и практически свойства:

- **Свойство при повдигане на степен:** Ако матрицата на съседство бъде повдигната на степен k , ненулевите стойности в получената матрица показват, че **съществува път с дължина k** между съответните върхове. Това важи както за ориентирани, така и за неориентирани графи.
- **Брой на пътищата:** Ако стойността на елемента $g[i, j]$ е броят на ребрата между два върха (включително примки), то положителната стойност в k -тата степен на матрицата показва **точния брой на всички различни пътища** с дължина k между тези два върха.
- **Памет и ефективност:** Реализацията изисква памет от порядъка на N^2 . Професорът подчертава, че това представяне **не е удобно за „разредени“ графи** (графи с малък брой ребра спрямо броя на върховете), тъй като матрицата заема излишно място.
- **Практическо приложение:** Този модел е удобен за алгоритми, при които броят на върховете е предварително известен и не е твърде голям.

Обхождане на граф в дълбочина и ширина

В Глава 6 от своето учебно помагало **проф. Ангел Голев** разглежда двата основни метода за системно посещение на всички върхове в един граф: обхождане в ширина (BFS) и обхождане в дълбочина (DFS).

1. Обхождане в ширина (Breadth-First Search - BFS)

Алгоритъм и логика: Обхождането в ширина започва от даден начален връх. Алгоритъмът намира всички негови наследници и ги обработва. Ключовата характеристика е, че следващият връх за обработка се взема от **най-рано обработените наследници**, което гарантира обхождане „ниво по ниво“ (или метод на вълната). За целта се използва структурата **опашка (Queue)**.

C# реализация (по проф. Голев): Професорът илюстрира BFS чрез задача за намиране на минимален път между думи. Използва се помощен клас `QueueElement`, за да се пази информация за върха, дълбочината и предходника (за възстановяване на пътя).

// Дефиниция на елемент в опашката

```
class QueueElement
{
    public int v { get; set; } // индекс на върха
    public int d { get; set; } // дълбочина
    public int p { get; set; } // индекс на предходника в опашката
}
```

// Основен цикъл на обхождането в ширина

```
bool SearchWordsPath(string w1, string w2)
{
    int k = FindWord(w1);
    if (k == -1) return false;

    mark[k] = true; // Масив за маркиране на посетени върхове
    my_queue.Enqueue(new QueueElement() { v = k, d = 1, p = -1 });

    int current_index = 0;
    while (current_index < my_queue.Count)
    {
        QueueElement q = my_queue.ElementAt(current_index++);
        foreach (int i in desc[q.v]) // Обхождане на наследниците (desc)
        {
            if (!mark[i])
            {
                mark[i] = true;
                my_queue.Enqueue(new QueueElement() { v = i, d = q.d + 1, p = current_index - 1 });
            }
        }
    }
}
```

```

        if (dict[i] == w2) // Проверка дали целта е достигната
        {
            return true;
        }
    }
}
return false;
}

```

2. Обхождане в дълбочина (Depth-First Search - DFS)

Алгоритъм и логика: При този метод се започва от даден връх, който се маркира като посетен. След това алгоритъмът търси **немаркиран наследник** и рекурсивно прилага същата процедура върху него, спускайки се колкото е възможно по-дълбоко в графа. Когато текущият връх няма повече непосетени наследници, алгоритъмът се връща към предходния връх (backtrack), за да провери за други негови наследници.

С# реализация (по проф. Голев): Професорът използва рекурсивна функция за реализацията. В примера по-долу се търси път между думи, но без гаранция, че той ще бъде минимален (за разлика от BFS).

```

// Рекурсивна функция за обхождане в дълбочина
bool DepthFirstSearch(int vert, string targetWord)
{
    mark[vert] = true; // Маркиране на текущия връх
    if (dict[vert] == targetWord) return true; // Проверка за край

    bool found = false;
    foreach (int v in desc[vert]) // За всеки наследник
    {
        if (!mark[v]) // Ако не е посетен
        {
            if (found = DepthFirstSearch(v, targetWord)) // Рекурсивно извикване
            {
                // При обратния ход може да се отпечата пътя
                Console.WriteLine(dict[v]);
                break;
            }
        }
    }
    return found;
}

```

Сравнение на методите според помагалото:

- **BFS (в ширина)** винаги намира **минималния път** (с най-малък брой ребра) между два върха в непретеглен граф.
- **DFS (в дълбочина)** е по-подходящ, когато е важно просто да се установи дали съществува път или трябва да се обхождат всички върхове при ограничена памет (не поддържа голяма опашка). Професорът отбелязва, че логиката на DFS е идентична с метода „обхождане с връщане назад“.

Минимален път в граф (Дейкстра)

Ето подробно изложение на **Тема 14 „Минимален път в граф (Дейкстра)“**, базирано на специализирания PDF файл за алгоритъма и книгата на Светлин Наков, с конкретна реализация от помагалото на проф. Голев (тъй като то предлага най-директния C# код за този конспект).

1. Логика на алгоритъма

Алгоритъмът на Дейкстра е **лаком (greedy) алгоритъм** за намиране на най-краткия път от един начален връх до всички останали върхове в претеглен граф с **неотрицателни тегла** на ребрата.

Основната идея е, че във всяка стъпка се избира върхът с най-малкото до момента „тентивно“ (временно) разстояние от източника. Тъй като теглата не са отрицателни, веднъж щом даден връх бъде „затворен“ (посетен), намереното разстояние до него се счита за окончателно и оптимално.

2. Стъпка по стъпка (Алгоритъм)

Според специализираните материали, процесът преминава през следните етапи:

1. **Инициализация:** На началния връх се присвоява разстояние **0**, а на всички останали – **безкрайност (infinity)**. Създава се списък с всички непосетени върхове.
2. **Избор на връх:** От списъка с непосетени върхове се избира този с **най-малкото текущо разстояние** (в началото това е стартовият връх).
3. **Релаксация (Обновяване на съседите):** За избрания връх се разглеждат всички негови непосетени съседни. Изчислява се разстоянието до всеки съсед през текущия връх. Ако това ново разстояние е по-малко от записаното до момента, стойността се актуализира.

4. **Маркиране:** След като се обхождат съседите, текущият връх се маркира като **посетен** и се премахва от списъка на непосетените. Той никога не се проверява повторно.
5. **Повторение:** Процесът се повтаря от стъпка 2, докато всички достижими върхове бъдат посетени или докато бъде достигнат целевият връх.

3. C# Реализация (по проф. Голев)

Реализацията използва **матрица на съседство** (`adjacencyMatrix`) и масив за разстоянията (`D`). За „безкрайност“ се ползва `int.MaxValue`.

```
private int[,] adjacencyMatrix; // Матрица на теглата
private int[] D;               // Намерени минимални пътища
private bool[] availableNodes; // Необходими върхове
public int NodesCount;        // Брой върхове

public void Solve(int node) // Търсене от начален връх 'node'
{
    // 1. Инициализация
    for (int i = 0; i < NodesCount; i++)
    {
        if (adjacencyMatrix[node, i] <= 0)
            D[i] = int.MaxValue;
        else
            D[i] = adjacencyMatrix[node, i];
        availableNodes[i] = true;
    }
    availableNodes[node] = false; // Маркираме началния като посетен

    while (true)
    {
        // 2. Намиране на необходим връх с минимално D[j]
        int j = -1;
        int di = int.MaxValue;
        for (int i = 0; i < NodesCount; i++)
        {
            if (availableNodes[i] && (D[i] < di))
            {
                di = D[i];
                j = i;
            }
        }

        if (j == -1) break; // Всички достижими върхове са обходени
    }
}
```

```

// 3. Релаксация
availableNodes[j] = false; // Маркираме текущия за обходен
for (int i = 0; i < NodesCount; i++)
{
    if (availableNodes[i] && (adjacencyMatrix[j, i] > 0))
    {
        // Проверка дали пътят през 'j' е по-кратък
        if (D[i] > (D[j] + adjacencyMatrix[j, i]))
        {
            D[i] = D[j] + adjacencyMatrix[j, i];
        }
    }
}
}
}

```

4. Сложност на алгоритъма

Времовата сложност зависи от използваните структури от данни за избор на минималния елемент:

- **При реализация с обикновен масив (като примера по-горе):** Сложността е $O(V^2)$, където V е броят на върховете. Това е ефективно за гъсти графи (с много ребра).
- **При реализация с приоритетна опашка (Binary Heap):** Сложността се подобрява до $O(E \log V)$, където E е броят на ребрата. Това е стандартът за разредени графи.
- **Теоретичен минимум:** С използване на **Fibonacci heap** сложността пада до $O(E + V \log V)$, което е асимптотично най-бързият известен вариант.

Важно ограничение: Алгоритъмът на Дейкстра **не работи** при наличие на **отрицателни тегла** на ребрата, тъй като неговата лакома логика предполага, че веднъж затворено разстояние не може да бъде подобрено (което е невярно, ако има отрицателни стойности).

Минимално покриващо дърво (лакоми алгоритми)

Въз основа на Глава 7 от учебното помагало на **проф. Ангел Голев**, ето подробно обяснение на Тема 15 „Минимално покриващо дърво (лакоми алгоритми)“.

1. Концепция за лакоми алгоритми (Greedy Algorithms)

Проф. Голев дефинира **лакомите (алчни) алгоритми** като техника за решаване на оптимизационни задачи. Основните характеристики на този подход според него са:

- **Последователно изграждане:** Решението се конструира стъпка по стъпка.
- **Локален избор:** На всяка стъпка се прави изборът, който изглежда най-добър в конкретния момент.
- **Риск и оптималност:** Професорът подчертава, че лакомите алгоритми невинаги гарантират оптимално решение, но са лесни за съставяне и работят бързо. Съществуват обаче задачи, като минималното покриващо дърво, за които този подход винаги намира глобалния оптимум.

2. Задача за Минимално покриващо дърво (MST)

Примерът на професора: Задачата се илюстрира с изграждането на пътна инфраструктура между населени места. Трябва да се определят пътища така, че от всяко място да може да се стигне до всяко друго, а общата дължина на всички изградени пътища да бъде минимална. **Математически модел:** Проблемът се представя чрез неориентиран граф с положителни тегла на ребрата. Търси се подграф-дърво, което включва всички върхове и сумата от теглата на ребрата му е минимална.

3. Алгоритъм на Прим (Prim's Algorithm)

Професорът посочва, че за тази задача лакомият алгоритъм (известен като алгоритъм на Прим) е винаги оптимален. **Логика стъпка по стъпка:**

1. Намира се **най-късото ребро** в целия граф и се добавя към дървото.
2. На всяка следваща стъпка се разглеждат всички ребра, които свързват върхове от дървото с върхове, които още не са в него.
3. Избира се това с **минимална дължина** и се добавя към дървото.
4. Процесът продължава, докато се включат всички върхове на графа.

4. C# реализация от проф. Голев

Реализацията използва матрица на съседство за графа, `HashSet<int>` за проследяване на включените върхове и списък от обекти `TEdge` за съхранение на ребрата на дървото.

```
public class MinimumSpanningTree {  
    int nodes_number;  
    int[,] my_graph;  
  
    class TEdge {  
        public int node1 { get; set; }  
        public int node2 { get; set; }  
        public int value { get; set; }  
    }  
}
```

```

HashSet<int> graph_nodes; // Върхове извън дървото
HashSet<int> tree_nodes; // Върхове в дървото
List<TEdge> tree_edges; // Ребра на MST

public MinimumSpanningTree() {
    // ... (Инициализация на графа и списъците) ...

    // 1. Намиране на най-късото ребро за старт
    int min = int.MaxValue;
    int n1 = -1, n2 = -1;
    for (int i = 0; i < nodes_number; i++)
        for (int j = i + 1; j < nodes_number; j++)
            if (my_graph[i, j] < min) {
                min = my_graph[i, j]; n1 = i; n2 = j;
            }

    tree_nodes.Add(n1); tree_nodes.Add(n2);
    tree_edges.Add(new TEdge() { node1 = n1, node2 = n2, value = my_graph[n1, n2] });
    graph_nodes.Remove(n1); graph_nodes.Remove(n2);

    // 2. Лакомо добавяне на следващи минимални ребра
    while (tree_nodes.Count < nodes_number) {
        min = int.MaxValue;
        foreach (int i in tree_nodes)
            foreach (int j in graph_nodes)
                if (my_graph[i, j] < min) {
                    min = my_graph[i, j]; n1 = i; n2 = j;
                }

        tree_nodes.Add(n2);
        tree_edges.Add(new TEdge() { node1 = n1, node2 = n2, value = my_graph[n1, n2] });
        graph_nodes.Remove(n2);
    }
    // ... (Отпечатване на резултатите) ...
}
}

```

Важно уточнение: Проф. Голев отбелязва, че използването на матрица на съседство е удобно за обяснение, но при реални задачи с „разредени“ графи (където ребрата са малко спрямо върховете) тя заема твърде много памет и следва да се използва друго представяне.

Топологична сортировка

Ето подробно изложение на **Тема 16 „Топологична сортировка“**, подготвено въз основа на книгата на Светлин Наков „Принципи на програмирането със C#“ и предоставените теоретични материали.

1. Идея на алгоритъма

Топологичната сортировка представлява линейно подреждане на върховете на един ориентиран граф по такъв начин, че за всяко ориентирано ребро (u, v) от върха u към върха v , върхът u да стои преди v в подредбата.

Визуално можем да си представим това като подреждане на всички върхове на графа по хоризонтална линия, така че всички ориентирани ребра да сочат само в една посока – отляво надясно.

2. Приложимост и условия

Алгоритъмът се прилага в ситуации, където имаме съвкупност от задачи или събития, между които съществуват зависимости (някои задачи трябва да бъдат изпълнени задължително преди други).

- **Тип графи:** Топологичната сортировка е дефинирана само за **ориентирани ациклични графи (DAG – Directed Acyclic Graphs)**.
- **Цикли:** Ако в графа съществува поне един цикъл (например път от u до v и обратно от v до u), топологична подредба е **невъзможна**, тъй като възниква противоречие в зависимостите.

3. Методи за реализация (по Наков)

В книгата са описани два основни подхода за решаване на задачата:

А. Метод чрез намиране на върхове без предшественици (Алгоритъм на Кан)

1. За всеки връх се изчислява неговата **входна степен** (брой ребра, които влизат в него).
2. Намира се връх, който няма предшественици (входна степен 0), и той се добавя в резултатната подредба.
3. Върхът се премахва от графа заедно с всички излизащи от него ребра.
4. Процесът се повтаря рекурсивно за останалия граф, докато всички върхове бъдат обработени.

Б. Метод чрез обхождане в дълбочина (DFS)

1. Изпълнява се стандартно **обхождане в дълбочина (DFS)** върху графа.

2. Всеки връх се добавя в списък (или стек) в момента, в който **напускаме рекурсията** за него (т.е. когато всички негови наследници са вече обходени).
3. Крайният резултат е списъкът, прочетен в **обратен ред**.

4. Сложност на алгоритъма

Топологичната сортировка е изключително ефективна:

- **Времева сложност:** $\Theta(V + E)$, където V е броят на върховете, а E – броят на ребрата. Това е линейна сложност, тъй като всяко ребро и всеки връх се посещават точно веднъж.
- **Сложност по памет:** Изисква допълнителна памет за съхранение на входните степени или списъка на посетените върхове.

5. Примерна реализация (DFS подход)

Според логиката на Наков, реализацията чрез DFS е елегантна, тъй като използва системния стек. Ето концептуален C# код:

```
void TopologicalSort(int v, bool[] visited, Stack<int> result)
{
    visited[v] = true; // Маркираме текущия за посетен

    foreach (int neighbor in adj[v]) // Обхождаме наследниците
    {
        if (!visited[neighbor])
        {
            TopologicalSort(neighbor, visited, result);
        }
    }

    // След като всички наследници са обработени, добавяме върха в стека
    result.Push(v);
}
```

Забележка: Ако по време на обхождането се срещне връх, който в момента е в стека на рекурсията (сив връх), това означава, че е открит **обратен път (цикъл)** и алгоритъмът трябва да съобщи за грешка.